



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE 3 Árboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	3ro – “B”
Alumnos participantes:	Curillo Pilamunga Diego Alexander
Asignatura:	Estructura de Datos
Docente:	Ing. José Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles.

Específicos:

- Analizar la eficiencia de los algoritmos recursivos en estructuras de datos no lineales para optimizar el tiempo de respuesta en operaciones de búsqueda y conteo.
- Estudiar el comportamiento de los Árboles Binarios de Búsqueda y Árboles N-arios mediante su implementación en C++ y Java para comprender la gestión de memoria en diferentes entornos.
- Desarrollar funciones de transformación y recorrido, como la inversión de nodos y el recorrido in-order, que permitan la manipulación dinámica de la jerarquía de datos.

2.2 Modalidad

Precencial.

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- **Inteligencia artificial, TAC**
- **PC de escritorio o portátil**
- **JDK instalado**
- **IDE de su elección.**
- **Git Bash**
- **GitHub**

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales



- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

De acuerdo con el tipo de trabajo, se plasmarán los resultados alcanzados en la guía práctica una vez ejecutadas las actividades. Se pueden emplear figuras y tablas las cuales deben ser numeradas.

2.7.1 Ejercicios C++

Ejercicio 1.cpp:

Mi TODO: Implementa tu lógica aquí

```
//TODO: Implementa tu lógica aquí
int contarNodos(NodoN* raiz) {
    // Si el nodo es nulo, no cuenta
    if (raiz == nullptr) {
        return 0;
    }

    // Empezamos contando 1 (el nodo actual)
    int total = 1;

    // Sumamos recursivamente la cantidad de nodos de cada hijo
    for (NodoN* hijo : raiz->hijos) {
        total += contarNodos(hijo);
    }

    return total;
}
```

Ejecución del Código:

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio1
Nodos calculados: 0
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> g++ Ejercicio1_Basico.cpp -o ejercicio1
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio1
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```



Ejercicio2.cpp:

Mi TODO: Implementa tu lógica aquí

```
//TODO: Implementa tu lógica aquí
Nodo* insertar(Nodo* raiz, int valor) {
    // Si el árbol está vacío, creamos el nuevo nodo
    if (raiz == nullptr) {
        return new Nodo(valor);
    }

    // Si el valor es menor, insertamos en el subárbol izquierdo
    if (valor < raiz->valor) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor);
    }
    // Si el valor es mayor, insertamos en el subárbol derecho
    else if (valor > raiz->valor) {
        raiz->derecho = insertar(raiz->derecho, valor);
    }

    return raiz;
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> g++ Ejercicio2_Binario.cpp -o ejercicio2
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio2
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
```

Ejercicio3.cpp:

Mi TODO: Implementa tu lógica aquí

```
// TODO: Implementa tu lógica aquí
int calcularAltura(Nodo* raiz) {
    // Caso base: si el nodo es nulo, la altura es 0
    if (raiz == nullptr) {
        return 0;
    }

    // Calculamos la altura de cada subárbol
    int alturaIzquierda = calcularAltura(raiz->izquierdo);
    int alturaDerecha = calcularAltura(raiz->derecho);

    // La altura del nodo actual es 1 más el máximo de sus hijos
    return 1 + max(alturaIzquierda, alturaDerecha);
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> g++ Ejercicio3_Binario.cpp -o ejercicio3
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio3
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de arbol nulo (esperado 0): 0
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> █
```



Ejercicio4.cpp

Mi TODO: Implementa tu lógica aquí

```
// TODO: Implementa tu logica de recorrido aquí
void inorderAux(Nodo* nodo, vector<int>& resultado) {
    // Caso base: si el nodo es nulo, terminamos la rama
    if (nodo == nullptr) {
        return;
    }

    // 1. Visitar subárbol izquierdo
    inorderAux(nodo->izquierdo, resultado);

    // 2. Procesar el nodo actual (Raíz)
    resultado.push_back(nodo->valor);

    // 3. Visitar subárbol derecho
    inorderAux(nodo->derecho, resultado);
}

vector<int> recorridoInOrder(Nodo* raiz) {
    vector<int> resultado;
    inorderAux(raiz, resultado);
    return resultado;
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> g++ Ejercicio4_Recorridos.cpp -o ejercicio4
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio4
--- Prueba Ejercicio 4 ---
Resultado esperado: 1 2 3 4 5 6 7
Tu resultado:      1 2 3 4 5 6 7
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp>
```

Ejercicio5.cpp

Mi TODO: Implementa tu lógica aquí

```
// TODO: Implementa tu lógica aquí
Nodo* invertir(Nodo* raiz) {
    // Caso base: si el nodo es nulo, no hay nada que invertir
    if (raiz == nullptr) {
        return nullptr;
    }

    // Intercambiamos los hijos del nodo actual usando una variable temporal
    Nodo* temporal = raiz->izquierdo;
    raiz->izquierdo = raiz->derecho;
    raiz->derecho = temporal;

    // Aplicamos la misma lógica recursivamente a los subárboles
    invertir(raiz->izquierdo);
    invertir(raiz->derecho);

    return raiz;
}
```



Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> g++ Ejercicio5_Transformacion.cpp -o ejercicio5
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp> ./ejercicio5
--- Prueba Ejercicio 5 ---
Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

Despues de invertir (Esperado: Izq 3 | Der 2):
Hijo Izq: 3 | Hijo Der: 2
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\cpp>
```

2.7.2 Ejercicios Java:

Ejercicio1.java

Mi TODO: Implementa tu lógica aquí

```
public class Ejercicio1_Basico {

    static int contarNodos(NodoN raiz) { //Quitamos el public para que la clase reciba y devuelva datos
        //TODO: Implementa tu lógica aquí. Pista usa recursividad
        // Caso base: si el nodo es nulo, no hay nodos que contar
        if (raiz == null) {
            return 0;
        }

        // Iniciamos el conteo en 1 (representando el nodo actual)
        int total = 1;

        // Usamos recursividad para sumar el conteo de todos los subárboles hijos
        for (NodoN hijo : raiz.hijos) {
            total += contarNodos(hijo);
        }

        return total;
    }
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> javac Ejercicio1_Basico.java
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> java Ejercicio1_Basico
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java>
```

Ejercicio2.java

Mi TODO: Implementa tu lógica aquí

```
public class Ejercicio2_Binario {

    // TODO: Implementa tu lógica aquí
    // Recuerda: menores a la izquierda, mayores o iguales a la derecha.
    static Nodo insertar(Nodo raiz, int valor) { //Quitamos el public para que la clase reciba y devuelva datos
        // Si la raíz es nula, creamos y retornamos el nuevo nodo
        if (raiz == null) {
            return new Nodo(valor);
        }

        // Lógica recursiva: menores a la izquierda, mayores o iguales a la derecha
        if (valor < raiz.valor) {
            raiz.izquierdo = insertar(raiz.izquierdo, valor);
        } else {
            // En este caso, el ejercicio pide mayores o iguales a la derecha
            raiz.derecho = insertar(raiz.derecho, valor);
        }

        // Retornamos la raíz (posiblemente con sus hijos actualizados)
        return raiz;
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JUNIO 2026



Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> javac Ejercicio2_Binario.java
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> java Ejercicio2_Binario
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo de Raiz (Esperado 5): 5
Hijo Derecho de Raiz (Esperado 15): 15
Hijo Izquierdo del 5 (Esperado 3): 3
```

Ejercicio3.java

Mi TODO: Implementa tu lógica aquí

```
// TODO: Implementa tu lógica aquí
public class Ejercicio3_Binario2 {
    static int calcularAltura(Nodo raiz) { //Quitamos el public para que la clase reciba y devuelva datos
        // Caso base: si el nodo es nulo, la altura es 0
        if (raiz == null) {
            return 0;
        }

        // Calculamos la altura de los subárboles izquierdo y derecho
        int alturaIzquierda = calcularAltura(raiz.izquierdo);
        int alturaDerecha = calcularAltura(raiz.derecho);

        // La altura actual es 1 (el nodo actual) más el máximo entre sus hijos
        return 1 + Math.max(alturaIzquierda, alturaDerecha);
    }
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> javac Ejercicio3_Binario2.java
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> java Ejercicio3_Binario2
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> S
```

Ejercicio4.java

Mi TODO: Implementa tu lógica aquí

```
public class RecorridoInOrder {
    // TODO: Implementa tu lógica de recorrido aquí
    static void inOrderAux(Nodo nodo, List<Integer> resultado) { //Quitamos el public para que la clase reciba y devuelva datos
        // Caso base: si el nodo es nulo, regresamos
        if (nodo == null) {
            return;
        }

        // 1. Visitar el subárbol izquierdo
        inOrderAux(nodo.izquierdo, resultado);

        // 2. Agregar el valor del nodo actual a la lista
        resultado.add(nodo.valor);

        // 3. Visitar el subárbol derecho
        inOrderAux(nodo.derecho, resultado);
    }

    static List<Integer> recorridoInOrder(Nodo raiz) { //Quitamos el public para que la clase reciba y devuelva datos
        List<Integer> resultado = new ArrayList<>();
        inOrderAux(raiz, resultado);
        return resultado;
    }
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> javac RecorridoInOrder.java
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> java RecorridoInOrder
--- Prueba Ejercicio 4 ---
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]
Tu resultado: [1, 2, 3, 4, 5, 6, 7]
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java>
```



Ejercicio5.java

Mi TODO: Implementa tu lógica aquí

```
public class Ejercicio5_Transformacion {  
    // TODO: Implementa tu lógica aquí  
    static Nodo invertir(Nodo raiz) { //Quitamos el public para que la clase reciba y devuelva datos  
        // Caso base: si el nodo es nulo, no hay nada que invertir  
        if (raiz == null) {  
            return null;  
        }  
  
        // Intercambiamos los hijos izquierdo y derecho del nodo actual  
        Nodo temporal = raiz.izquierdo;  
        raiz.izquierdo = raiz.derecho;  
        raiz.derecho = temporal;  
  
        // Llamamos a la función recursivamente para invertir los subárboles  
        invertir(raiz.izquierdo);  
        invertir(raiz.derecho);  
  
        return raiz;  
    }  
}
```

Ejecución del Código

```
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> javac Ejercicio5_Transformacion.java  
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> java Ejercicio5_Transformacion  
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2  
PS C:\Users\Usuario\Desktop\3ro\Estructura de Datos\APE_Arboles\java> |
```

2.8 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

La implementación de estos ejercicios permitió validar que la recursividad es la herramienta más efectiva para navegar estructuras jerárquicas complejas de forma clara. Se logró unificar la lógica de programación entre C++ y Java, demostrando que los principios de las estructuras de datos son universales independientemente del lenguaje utilizado.

A través de las pruebas unitarias en la terminal, se confirmó que el manejo correcto de punteros y referencias es crítico para evitar errores de acceso a memoria. El dominio de estas técnicas es fundamental para el desarrollo de software robusto y eficiente en la resolución de problemas de ingeniería.

2.10 Recomendaciones



Se sugiere profundizar en el estudio de los modificadores de acceso en Java para prevenir conflictos de visibilidad al exponer tipos no públicos en métodos de clase. Asimismo, se recomienda practicar la depuración de punteros en C++ para garantizar la correcta liberación de memoria en estructuras dinámicas.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

2.12 Anexos

Clonado de Repositorio desde la Bash

```
Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ git clone "/c:/Users/Usuario/Desktop/3ro/Estructura de Datos/APE_Arboles"
Cloning into 'Estructura de Datos'...
fatal: 'C:/Users/Usuario/Desktop/3ro/Estructura de Datos' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ git clone "/c:/Users/Usuario/Desktop/3ro/Estructura de Datos"
Cloning into 'Estructura de Datos'...
fatal: 'C:/Users/Usuario/Desktop/3ro/Estructura de Datos' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ git clone "C:/Users/Usuario/Desktop/3ro/Estructura de Datos"
Cloning into 'Estructura de Datos'...
fatal: 'C:/Users/Usuario/Desktop/3ro/Estructura de Datos' does not appear to be
a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ git clone https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git
Cloning into 'Ape-3-Estructura-de-Datos'...
remote: Repository not found.
fatal: repository 'https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git/' not found

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ cd "/c:/Users/Usuario/Desktop/3ro/Estructura de Datos"

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos
$ cd "/c:/Users/Usuario/Desktop/3ro/Estructura de Datos/APE_Arboles"

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles
$ git init
Initialized empty Git repository in C:/Users/Usuario/Desktop/3ro/Estructura de Datos/APE_Arboles/.git/

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git remote agregar origen https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git
git: 'remote' is not a git command. See 'git --help'.

The most similar command is
remote

11 files changed, 372 insertions(+)
create mode 100644 README.md
create mode 100644 cpp/Ejercicio1_Basico.cpp
create mode 100644 cpp/Ejercicio2_Binario.cpp
create mode 100644 cpp/Ejercicio3_Binario.cpp
create mode 100644 cpp/Ejercicio4_Recorridos.cpp
create mode 100644 cpp/Ejercicio5_Transformacion.cpp
create mode 100644 java/Ejercicio1_Basico.java
create mode 100644 java/Ejercicio2_Binario.java
create mode 100644 java/Ejercicio3_Binario2.java
create mode 100644 java/Ejercicio5_Transformacion.java
create mode 100644 java/RecorridoInOrder.java

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git'

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git push -u origin master
remote: Repository not found.
fatal: repository 'https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git/' not found

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git remote remove origin

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git remote add origin https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (master)
$ git branch -M main

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (main)
$ git push -u origin main
remote: Repository not found.
fatal: repository 'https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git/' not found

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (main)
$ git remote remove origin

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (main)
$ git remote add origin https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git

Usuario@DESKTOP-8GU883A MINGW64 ~/Desktop/3ro/Estructura de Datos/APE_Arboles (main)
$ git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (15/15), 5.19 KiB | 2.59 MiB/s, done.
Total 15 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Remote: Resolving deltas: 100% (2/2), done.
To https://github.com/Diego02255/Ape-3-Estructura-de-Datos.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.
```




UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

CICLO ACADÉMICO: ENERO – JUNIO 2026



Vista del README de la actividad realizada

The screenshot shows a GitHub repository page for 'Ape-3-Estructura-de-Datos' by user 'Diego2255'. The README is titled 'Estructuras de Datos - Resolución de Ejercicios (Árboles)'. It describes a repository containing solutions for practical exercises on non-linear data structures, specifically Binary Search Trees (BST). The exercises are implemented in C++ and Java, applying concepts like recursion, pointers, and memory management. The 'Contenido de los Ejercicios' section lists 'Ejercicio 1: Conteo de Nodos (Árbol N-ario)' with a description of a recursive function to count nodes and a list of concepts: Recursividad, Listas dinámicas (vector in C++, List in Java), and Captura de Ejecución Cpp.

Vista del Visual Studio Code

The screenshot shows the Visual Studio Code editor with a project named 'APE_Arboles'. The file explorer on the left shows a directory structure with files for 'Ejercicio5_Transformacion.cpp', 'Ejercicio5_Transformacion.class', 'Ejercicio5_Transformacion.java', and 'Ejercicio5_Transformacion.class'. The main editor window displays the 'Ejercicio5_Transformacion.java' file, which contains a recursive function 'invertir(Nodo raiz)' that swaps the left and right children of a binary tree node. The function is implemented in Java, with comments in Spanish explaining the logic. The 'main' method creates a binary tree with root node 1, left child 2, and right child 3, and then calls the 'invertir' function to invert the tree. The output of the program is shown in the terminal at the bottom, displaying the tree structure before and after inversion.